

Attention Is All You Need

<https://arxiv.org/abs/1706.03762>

Abstract

- 기존 sequence model은 주로 RNN이나 CNN을 기반으로 구성되었다.
- RNN은 이전 위치의 계산이 끝나야 다음 위치를 계산할 수 있어 병렬 처리가 어렵다.
- 논문은 recurrence와 convolution을 제거하고, attention만으로 구성된 Transformer를 제안한다.
- Transformer는 self-attention을 통해 모든 token 사이의 관계를 직접 계산하며, 입력 sequence를 병렬적으로 처리한다.
- Machine translation에서 기존 모델보다 높은 성능을 기록하면서 학습 비용도 줄였다.

1. Introduction

1. 기존 RNN의 한계

RNN의 현재 hidden state는 이전 hidden state에 의존한다.

$$h_t = f(h_{t-1}, x_t)$$

- 이전 위치의 계산이 끝나야 다음 위치를 계산할 수 있다.
- 하나의 sequence 내부 연산을 병렬화하기 어렵다.
- 멀리 떨어진 token 사이의 정보가 여러 hidden state를 거쳐 전달된다.
- Sequence가 길어질수록 long-range dependency 학습이 어려워질 수 있다.

2. CNN 기반 모델의 한계

CNN은 여러 위치를 병렬적으로 계산할 수 있다.

하지만 한 convolution layer는 kernel 주변의 제한된 영역만 참고한다. 따라서 멀리 떨어진 token을 연결하려면 여러 layer를 쌓아야 한다.

3. Transformer의 제안

Transformer는 RNN과 CNN을 제거하고 attention만으로 sequence를 처리한다.

핵심 장점:

- 모든 입력 token을 병렬적으로 계산
- 한 번의 self-attention으로 모든 token을 직접 연결
- 멀리 떨어진 token 사이의 짧은 정보 전달 경로
- 기존 recurrent model보다 빠른 학습

다만 실제 문장을 생성하는 inference 과정에서는 이전에 생성한 token을 이용하므로 순차적으로 동작한다.

2. Background

Self-Attention

Self-attention은 하나의 sequence에 있는 token들이 서로의 정보를 참고하도록 하는 연산이다.

각 token은 다른 token과의 관련도를 계산하고, 관련도가 높은 token의 정보를 더 많이 반영한다.

예시:

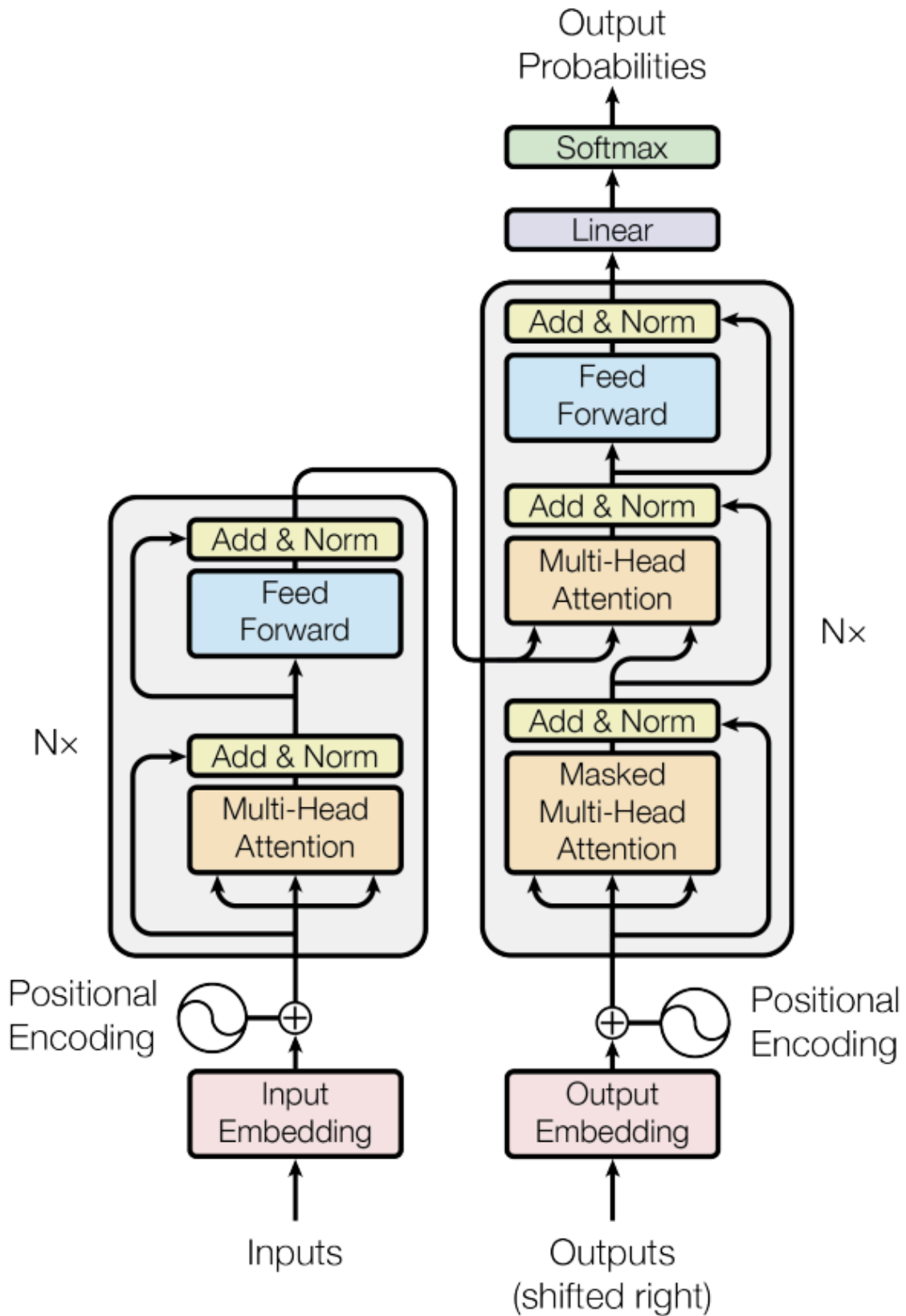
| The animal did not cross the street because it was tired.

- `it`의 의미를 이해하려면 앞에 있는 `animal`을 참고해야 한다.
- Self-attention은 `it`이 `animal`에 높은 attention weight를 주도록 학습할 수 있다.

기존 모델에서도 attention은 사용되었지만, 주로 RNN 기반 encoder와 decoder를 연결하는 보조적인 역할이었다.

Transformer는 attention을 sequence representation을 만드는 핵심 구조로 사용한다.

3. Model Architecture



Transformer는 encoder-decoder 구조를 사용한다.

전체 흐름: Input Embedding + Positional Encoding → Encoder → Decoder → Linear Layer → Softmax → 다음 token 예측

3.1 Encoder and Decoder Stacks

1. Encoder

Encoder는 동일한 구조의 layer 6개를 쌓아 구성한다.

각 encoder layer:

1. Multi-Head Self-Attention
2. Position-wise Feed-Forward Network

전체 구조:

Input → Multi-Head Self-Attention → Add & Norm → Feed-Forward Network → Add & Norm → Output

Multi-Head Self-Attention

- 각 입력 token이 모든 입력 token을 참고한다.
- Token 자체의 정보와 전체 문맥이 반영된 representation을 만든다.

Feed-Forward Network

- Self-attention의 결과를 token별로 변환한다.
- 같은 layer에서는 모든 token 위치에 동일한 parameter를 사용한다.

Residual Connection과 Layer Normalization

각 sub-layer에는 residual connection과 Layer Normalization을 적용한다.

$$\text{LayerNorm}(x + \text{Sublayer}(x))$$

Residual addition을 위해 모든 sub-layer의 입출력 차원은 다음과 같이 유지한다.

$$d_{\text{model}} = 512$$

2. Decoder

Decoder 역시 동일한 구조의 layer 6개를 쌓아 구성한다.

각 decoder layer:

1. Masked Multi-Head Self-Attention
2. Encoder-Decoder Attention
3. Position-wise Feed-Forward Network

Masked Self-Attention

Decoder는 현재 위치보다 뒤에 있는 미래 token을 참고할 수 없다.

미래 위치의 attention score를 매우 작은 값으로 바꾸면 Softmax 이후 해당 위치의 attention weight가 0에 가까워진다.

학습할 때 정답 sequence는 한 칸 오른쪽으로 이동하여 decoder에 입력한다.

- 정답: I love football
- Decoder input: <START>, I, love
- 예측 target: I, love, football

Encoder-Decoder Attention

- Query는 decoder에서 만들어진다.
- Key와 Value는 encoder의 최종 output에서 만들어진다.
- Decoder가 다음 token을 생성할 때 입력 sequence의 관련 위치를 참고한다.

3.2 Attention

Attention은 Query와 Key의 관련도를 계산하고, 그 결과를 이용해 Value를 가중합하는 연산이다.

- Query: 현재 token이 찾는 정보
- Key: 각 token과의 관련도를 비교하는 기준
- Value: 실제로 가져올 정보

입력 representation을 x 라고 하면 다음과 같이 계산한다.

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V$$

- W^Q, W^K, W^V 는 모두 학습되는 parameter이다.
- 각 token마다 서로 다른 Query, Key, Value가 생성된다.

- 이 vector들을 행렬로 묶어 한 번에 attention을 계산한다.

Key와 Value를 분리하는 이유

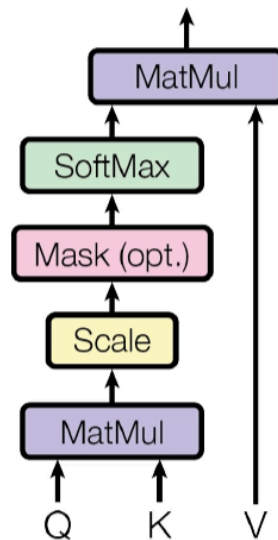
- Key는 Query와의 관련도를 계산한다.
- Value는 관련도가 높은 위치에서 실제 정보를 가져온다.

따라서 모델은 다음 두 역할을 구분하여 학습할 수 있다.

- 어떤 위치를 참고할 것인가?
- 해당 위치에서 어떤 정보를 가져올 것인가?

3.2.1 Scaled Dot-Product Attention

Scaled Dot-Product Attention



기본식:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

계산 과정:

1. Query와 모든 Key의 관련도를 계산한다.

$$\text{Score} = QK^T$$

2. Score를 scaling한다.

$$\text{ScaledScore} = \frac{QK^T}{\sqrt{d_k}}$$

3. Softmax로 attention weight를 계산한다.

$$A = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right)$$

4. Attention weight를 이용해 Value를 가중합한다.

$$\text{Output} = AV$$

Scaling을 하는 이유

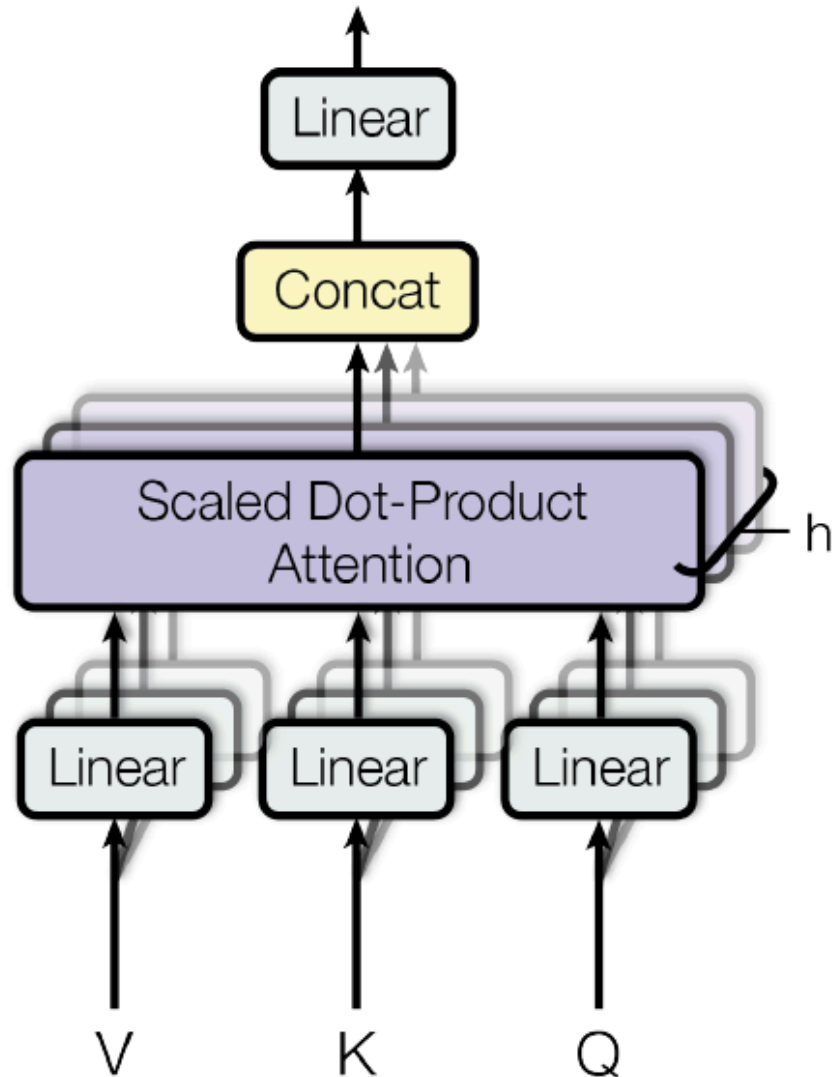
`d_k`가 커질수록 dot product의 크기도 커질 가능성이 높다.

Score가 지나치게 커지면 Softmax가 특정 위치에 확률을 과도하게 집중하고 gradient가 작아질 수 있다.

따라서 Score를 `sqrt(d_k)`로 나누어 값의 크기를 조절한다.

3.2.2 Multi-Head Attention

Multi-Head Attention



하나의 attention head만 사용하면 서로 다른 종류의 관계가 하나의 attention 결과에 섞일 수 있다.

Multi-Head Attention은 Q, K, V 를 여러 표현 공간으로 투영한 뒤 각 공간에서 attention을 병렬적으로 수행한다.

각 head:

$$\text{head}_i = \text{Attention} \left(QW_i^Q, KW_i^K, VW_i^V \right)$$

전체 출력:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

논문의 기본 설정:

$$d_{\text{model}} = 512, \quad h = 8, \quad d_k = d_v = \frac{d_{\text{model}}}{h} = 64$$

8개 head의 결과를 concatenate하면 다시 512차원이 된다.

$$8 \times 64 = 512$$

Multi-Head Attention의 장점:

- 서로 다른 위치에 동시에 집중할 수 있다.
 - 서로 다른 표현 공간에서 다양한 관계를 학습할 수 있다.
 - 각 head의 역할은 사람이 지정하지 않고 학습 과정에서 결정된다.
-

3.2.3 Transformer의 세 가지 Attention

Encoder Self-Attention

- Q, K, V가 모두 encoder의 이전 layer output에서 생성된다.
- 각 입력 token이 모든 입력 token을 참고한다.

Decoder Masked Self-Attention

- Q, K, V가 모두 decoder에서 생성된다.
- 미래 token은 mask를 통해 차단한다.

Encoder-Decoder Attention

- Q는 decoder에서 생성된다.
 - K와 V는 encoder에서 생성된다.
 - 출력 token을 만들 때 입력 sequence의 관련 정보를 참고한다.
-

3.3 Position-wise Feed-Forward Network

각 encoder와 decoder layer에는 Feed-Forward Network가 포함된다.

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

차원 변화:

$$512 \rightarrow 2048 \rightarrow 512$$

- 첫 번째 linear layer에서 차원을 확장한다.
- ReLU로 비선형성을 추가한다.
- 두 번째 linear layer에서 다시 512차원으로 줄인다.

Self-Attention과 Feed-Forward Network의 역할 구분:

- Self-Attention: 서로 다른 token 사이의 정보를 결합한다.
- Feed-Forward Network: 각 token vector 내부의 feature를 변환한다.

같은 layer에서는 모든 token 위치에 동일한 FFN parameter를 사용한다.

3.4 Embedding and Softmax

Embedding layer는 token을 `d_model` 차원의 vector로 변환한다.

논문에서는 다음 세 부분의 weight를 공유한다.

- Encoder input embedding
- Decoder input embedding
- Decoder output의 pre-softmax linear layer

Decoder의 최종 output은 linear layer를 통해 vocabulary 크기의 logit으로 변환된다.

Softmax는 이 logit을 다음 token에 대한 확률로 변환한다.

Embedding output에는 다음 scaling을 적용한다.

$$\text{ScaledEmbedding} = \text{Embedding} \times \sqrt{d_{\text{model}}}$$

3.5 Positional Encoding

Transformer는 token을 순차적으로 처리하지 않기 때문에 별도의 위치 정보가 필요하다.

입력 representation은 다음과 같이 구성된다.

$$\text{InputRepresentation} = \text{TokenEmbedding} + \text{PositionalEncoding}$$

논문은 sine과 cosine 기반의 고정 positional encoding을 사용한다.

짝수 차원:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

홀수 차원:

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

- **pos** : token의 위치
- **i** : positional encoding의 차원
- 각 차원은 서로 다른 주기를 사용한다.
- 이를 통해 각 token 위치가 서로 다른 pattern을 가진다.

학습 가능한 positional embedding도 비슷한 성능을 보였다.

따라서 중요한 것은 sine과 cosine 자체가 아니라, recurrence가 없는 구조에 위치 정보를 추가하는 것이다.

4. Why Self-Attention

Self-attention을 사용하는 주요 이유는 다음과 같다.

1. 병렬 처리

RNN과 달리 모든 token 위치를 동시에 계산할 수 있다.

2. 짧은 정보 전달 경로

Self-attention에서는 모든 token이 한 layer에서 직접 연결된다.

Maximum path length:

$$\text{Self-Attention} : O(1), \quad \text{RNN} : O(n)$$

CNN은 멀리 떨어진 token을 연결하려면 여러 layer가 필요하다.

따라서 self-attention은 long-range dependency를 학습하는 데 유리하다.

3. 계산 복잡도

$$\text{Self-Attention} : O(n^2d), \quad \text{RNN} : O(nd^2)$$

n 이 sequence length이고 d 가 representation dimension일 때 다음 조건에서는 self-attention의 계산량이 더 작다.

$$n < d$$

한계

Self-attention은 모든 token 쌍을 비교한다.

따라서 sequence가 길어질수록 attention matrix의 계산량과 memory 사용량이 다음과 같이 증가한다.

$$O(n^2)$$

5. Training

- WMT 2014 English-German과 English-French translation dataset을 사용했다.
- Tokenization에는 subword 기반의 Byte-Pair Encoding을 사용했다.
- Optimizer는 Adam을 사용했다.
- 초기에는 learning rate를 증가시키는 warmup을 적용하고, 이후에는 점차 감소시켰다.
- Regularization에는 dropout과 label smoothing을 사용했다.

6. Experiments

Machine Translation

- English-German translation에서 Transformer는 28.4 BLEU를 기록했다.
- 당시 기존 최고 성능보다 2 BLEU 이상 높은 결과였다.
- English-French translation에서도 기존 모델보다 적은 학습 비용으로 높은 성능을 기록했다.

Model Analysis

- Multi-Head Attention이 single-head attention보다 좋은 성능을 보였다.
- 모델 크기를 증가시키면 대체로 성능이 향상되었다.
- Sinusoidal positional encoding과 learned positional embedding은 비슷한 성능을 보였다.

Constituency Parsing

- Transformer를 translation 이외의 parsing task에도 적용했다.
 - 별도의 큰 구조 변경 없이도 경쟁력 있는 결과를 기록했다.
-

7. Conclusion

- Transformer는 RNN과 CNN 없이 attention을 중심으로 sequence를 처리한다.
- 병렬 학습이 가능하고, 멀리 떨어진 token 사이의 관계를 직접 계산할 수 있다.
- Translation과 parsing 실험을 통해 이러한 구조가 실제로 효과적임을 확인했다.